

Hilos en Linux

Creación, sincronización y cancelación de hilos con pthreads



```
$ gcc -o tc thread-create.c -  
lpthread  
$ ./tc  
  
xxxxxxxxxxxxxxxxxx  
oxxxxxxxxxxxxxxxx  
xooooooooxxxxxxxx  
^C
```

Agenda

1

Creación y ciclo de vida

pthread_create, join, atributos y valores de retorno

§4.1

2

Cancelación de hilos

Estados de cancelación y secciones críticas

§4.2

3

Datos específicos de hilo

Aislar variables por hilo y cleanup handlers

§4.3

4

Sincronización

Condiciones de carrera, mutexes, semáforos y variables de condición

§4.4

5

Implementación y comparación

Hilos como procesos en GNU/Linux; procesos vs. hilos

§4.5–4.6

¿Qué es un hilo?

- Un hilo es una unidad de ejecución más fina que un proceso: vive dentro de un proceso.
- Todos los hilos de un proceso comparten el mismo espacio de memoria, descriptores de archivo y demás recursos.
- La API es pthreads — se declara en `<pthread.h>` y se linkea agregando `-lpthread`.

`fork()`

- El hijo recibe una copia de la memoria, descriptores y demás recursos.
- Padre e hijo evolucionan de forma independiente a partir de ahí.
- Compartir datos entre ellos requiere IPC (Capítulo 5).

`pthread_create()`

- El nuevo hilo comparte la misma memoria, archivos y recursos del original.
- Si un hilo modifica una variable global, los demás la ven al instante.
- Si cualquier hilo llama a una función `exec`, todos los demás hilos terminan.

pthread_create(): el punto de partida

```
int pthread_create(pthread_t*, const pthread_attr_t*, void* (*)(void*), void*);
```

1

Dónde se guarda el ID del nuevo hilo
`pthread_t* thread`

2

Atributos del hilo (NULL = valores por defecto)
`pthread_attr_t* attr`

3

La función que el hilo va a ejecutar
`void* (*)(void*)`

4

El argumento que recibe esa función
`void* arg`



```
pthread_create(&thread_id, NULL, &print_xs, NULL);
```

thread-create.c: el primer hilo

```
// gcc -o thread-create thread-create.c -lpthread

#include <pthread.h>
#include <stdio.h>

void* print_xs(void* unused) {
    while (1)
        fputc('x', stderr);
    return NULL;
}

int main(void) {
    pthread_t thread_id;
    pthread_create(&thread_id, NULL, &print_xs, NULL);
    while (1)
        fputc('o', stderr);
    return 0;
}
```

Pasar datos a un hilo

- El argumento `void*` permite pasar un puntero a una estructura con los datos del hilo.
- Así se puede reusar la misma función para varios hilos, cada uno con los parámetros apropiados.
- Ejemplo del libro: `char_print()`, reusada por dos hilos con distinto carácter y contador.

```
struct char_print_parms {  
    char character;  
    int count;  
};
```

✗ Peligro

Si `main()` (o la función que instancia los hilos) termina antes que los hilos, la memoria de sus argumentos —variables locales del `stack`— puede liberarse mientras el hilo todavía la está usando.

→ La solución está en la próxima diapositiva: `pthread_join()`

Esperar hilos: `pthread_join()`

- Así como `wait()` espera a que termine un proceso hijo, `pthread_join()` espera a que termine un hilo.
- Recibe el ID del hilo y un puntero `void*` donde se guarda su valor de retorno (NULL si no interesa).
- Resuelve el peligro de la diapositiva anterior: `main()` no sigue hasta que los hilos ya no necesitan sus argumentos.
- Un hilo joinable no libera sus recursos hasta que alguien lo joina — igual que un proceso zombie espera a `wait()`.



join corregido: sin punteros colgantes

```
int main(void) {
    pthread_t thread1_id, thread2_id;
    struct char_print_parms thread1_args, thread2_args;

    thread1_args.character = 'x';
    thread1_args.count = 30000;
    pthread_create(&thread1_id, NULL, &char_print, &thread1_args);

    thread2_args.character = 'o';
    thread2_args.count = 20000;
    pthread_create(&thread2_id, NULL, &char_print, &thread2_args);

    // Esperar a que ambos hilos terminen antes de liberar sus args.
    pthread_join(thread1_id, NULL);
    pthread_join(thread2_id, NULL);

    return 0;
}
```

Valor de retorno e identidad del hilo

- El segundo argumento de `pthread_join` recibe el valor de retorno del hilo (tipo `void*`).
- Es el valor que devuelve la función del hilo, o el que se pasa a `pthread_exit()`.
- `pthread_self()` devuelve el ID del hilo actual; `pthread_equal()` compara dos IDs.
- Por ejemplo, sirve para evitar que un hilo intente hacerse join a sí mismo (error `EDEADLK`).

```
int prime;
pthread_join(thread, (void*) &prime);
printf("El %d-esimo primo es %d.\n", n, prime);
```

Atributos: joinable vs. detached

Joinable (por defecto)

- No se libera solo al terminar.
- Otro hilo debe llamar `pthread_join()` para liberar sus recursos.
- Mientras nadie lo joinea, es como un hilo 'zombie'.

Detached

- Se limpia automáticamente al terminar.
- Se activa con `pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED)`.
- Ya no se puede hacer join ni obtener su valor de retorno.

`pthread_detach()` convierte un hilo joinable en detached en cualquier momento — pero no al revés.

`pthread_cancel()`: terminar un hilo desde afuera

- `pthread_cancel(id)` le pide a otro hilo que termine.
- Igual conviene joinarlo después, para liberar sus recursos (salvo que sea detached). Su valor de retorno especial es `PTHREAD_CANCELED`.

Asíncrono

Puede cancelarse en cualquier punto de su ejecución.

Síncrono (por defecto)

Solo se cancela al llegar a un punto de cancelación.

No cancelable

Ignora silenciosamente los pedidos de cancelación.

```
pthread_testcancel()
```

Crea un punto de cancelación explícito — útil en cálculos largos y sin otras llamadas cancelables.

Una sección crítica no cancelable

```
  
// Transferir DOLLARS de FROM_ACCT a TO_ACCT.  
int process_transaction(int from_acct, int to_acct, float dollars) {  
    int old_state;  
  
    if (account_balances[from_acct] < dollars)  
        return 1;  
  
    // Inicio de la seccion critica: no cancelable.  
    pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, &old_state);  
  
    account_balances[to_acct] += dollars;  
    account_balances[from_acct] -= dollars;  
  
    // Fin de la seccion critica: restaura el estado anterior.  
    pthread_setcancelstate(old_state, NULL);  
  
    return 0;  
}
```

Una copia por hilo, y limpieza al salir

Datos específicos de hilo

- Cada hilo puede tener su propia copia de una variable — por ejemplo, un archivo de log por hilo.
- `pthread_key_create()` crea una clave compartida por todos los hilos.
- Cada hilo lee/escribe su copia con `pthread_getspecific()` / `pthread_setspecific()`.

Cleanup handlers

- Una función que se ejecuta cuando el hilo termina o es cancelado.
- Se registra con `pthread_cleanup_push()` y se retira con `pthread_cleanup_pop()`.
- En C++: conviene lanzar una excepción propia en vez de llamar `pthread_exit()` directo, para que se destruyan los objetos locales.

Condiciones de carrera

Dos hilos pueden leer `job_queue` al mismo tiempo y tomar el mismo trabajo:

```
struct job {
    struct job* next;
    // ...datos del trabajo...
};

struct job* job_queue;

void* thread_function(void* arg) {
    while (job_queue != NULL) {
        struct job* next_job = job_queue;
        job_queue = job_queue->next; // riesgo: 2 hilos leen esto a la vez
        process_job(next_job);
        free(next_job);
    }
    return NULL;
}
```

Mutexes: exclusión mutua

- Un mutex es un candado que solo un hilo puede tener a la vez.
- Como el candado del baño: quien llega primero lo tranca; el resto espera afuera.
- PTHREAD_MUTEX_INITIALIZER inicializa un mutex global sin llamar a pthread_mutex_init().
- pthread_mutex_unlock() debe llamarlo el mismo hilo que lo tomó.

```
pthread_mutex_lock
```

Toma el candado; bloquea si otro hilo ya lo tiene.

```
pthread_mutex_unlock
```

Libera el candado.

```
pthread_mutex_trylock
```

Intenta tomarlo sin bloquear; devuelve EBUSY si está ocupado.

La cola de trabajos, protegida por un mutex

```
struct job* job_queue;
pthread_mutex_t job_queue_mutex = PTHREAD_MUTEX_INITIALIZER;

void* thread_function(void* arg) {
    while (1) {
        struct job* next_job;
        pthread_mutex_lock(&job_queue_mutex);
        if (job_queue == NULL)
            next_job = NULL;
        else {
            next_job = job_queue;
            job_queue = job_queue->next;
        }
        pthread_mutex_unlock(&job_queue_mutex);

        if (next_job == NULL) break;
        process_job(next_job);
        free(next_job);
    }
    return NULL;
}
```

Tipos de mutex y deadlocks simples

Fast (por defecto)

Si el mismo hilo lo bloquea dos veces seguidas, se genera un deadlock consigo mismo.

Rekursivo

El mismo hilo puede bloquearlo varias veces; debe desbloquearlo la misma cantidad de veces.

Error-checking

GNU/Linux detecta el doble bloqueo y devuelve el error EDEADLK, en vez de trabarse.

→ Prueba sin bloquear

`pthread_mutex_trylock()` intenta tomar el mutex sin esperar: si está ocupado devuelve EBUSY de inmediato, en vez de bloquear al hilo.

Semáforos

- Un semáforo es un contador no negativo para sincronizar hilos.
- `sem_wait()` decrementa el contador; si ya es cero, bloquea hasta que otro hilo haga `sem_post()`.
- `sem_post()` incrementa el contador y despierta a un hilo bloqueado, si hay alguno.
- A diferencia de un mutex, un semáforo 'recuerda' los avisos aunque nadie esté esperando en ese momento.



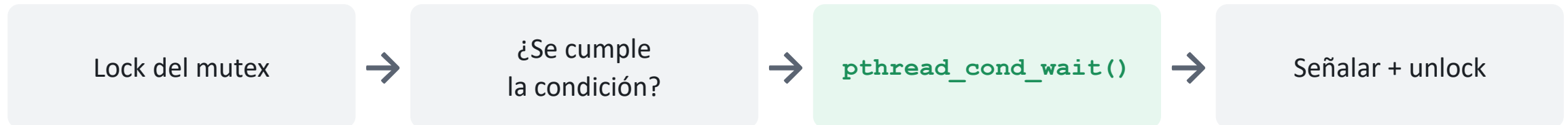
```
sem_t job_queue_count;
sem_init(&job_queue_count, 0, 0);

// Hilo trabajador:
sem_wait(&job_queue_count);

// Al encolar un trabajo:
sem_post(&job_queue_count);
```

Variables de condición

- Permiten que un hilo espere hasta que otro le avise que una condición cambió.
- Siempre se usan junto a un mutex, para evitar una condición de carrera entre 'revisar' y 'esperar'.
- `pthread_cond_signal()` despierta a un hilo esperando; `pthread_cond_broadcast()` los despierta a todos.



Un hilo controlado por su condición

```
int thread_flag;
pthread_cond_t thread_flag_cv;
pthread_mutex_t thread_flag_mutex;

void* thread_function(void* arg) {
    while (1) {
        pthread_mutex_lock(&thread_flag_mutex);
        while (!thread_flag)
            pthread_cond_wait(&thread_flag_cv, &thread_flag_mutex);
        pthread_mutex_unlock(&thread_flag_mutex);
        do_work();
    }
}

void set_thread_flag(int flag_value) {
    pthread_mutex_lock(&thread_flag_mutex);
    thread_flag = flag_value;
    pthread_cond_signal(&thread_flag_cv);
    pthread_mutex_unlock(&thread_flag_mutex);
}
```

Deadlocks entre múltiples hilos

Ocurre cuando dos o más hilos esperan, cada uno, algo que solo el otro puede provocar:

Hilo A

Toma Mutex 1, después pide Mutex 2 — queda bloqueado.

Hilo B

Toma Mutex 2, después pide Mutex 1 — queda bloqueado.

→ Solución

Todos los hilos deben bloquear los mismos recursos siempre en el mismo orden — no solo mutexes: también archivos o dispositivos.

Hilos = procesos en GNU/Linux

```
void* thread_function(void* arg) {
    fprintf(stderr, "pid del hijo: %d\n",
        (int) getpid());
    while (1);
}

int main(void) {
    pthread_t thread;
    fprintf(stderr, "pid principal: %d\n",
        (int) getpid());
    pthread_create(&thread, NULL,
        &thread_function, NULL);
    while (1);
}
```

\$ ps x

14608

hilo principal (main)

14609

hilo manejador — interno, de
GNU/Linux

14610

hilo creado con pthread_create()

- pthread_create() crea internamente un nuevo proceso Linux que comparte memoria con el original.

Señales y la `syscall clone()`

- Como cada hilo es, por debajo, un proceso distinto, una señal externa no es ambigua: llega al hilo principal.
- `pthread_kill(id, señal)` envía una señal a un hilo específico dentro del mismo programa.
- `clone()` es la `syscall` genérica detrás de `fork()` y `pthread_create()`: permite elegir qué se comparte y qué se copia.
- En la práctica no se usa `clone()` directamente — se usa `fork()` o `pthread_create()` según el caso.

```
pthread_kill(id, SIGUSR1)
```

Envía una señal a un hilo específico — no a todo el proceso ni a los demás hilos.

→ El hilo común del capítulo

`fork()`, `exec()` y `pthread_create()` son, en el fondo, distintas formas de llamar a la misma `syscall`: `clone()`.

Procesos vs. Hilos

Procesos

- Cada uno tiene su propia copia de la memoria.
- Puede ejecutar un programa distinto con `exec()`.
- Un proceso con errores no afecta a otros.
- Compartir datos entre ellos requiere IPC (Capítulo 5).

Hilos

- Comparten memoria, archivos y demás recursos.
- Todos corren el mismo programa.
- Un hilo con errores puede corromper a los demás.
- Compartir datos es trivial — pero exige cuidar las condiciones de carrera.

Regla práctica: hilos para paralelismo fino (mismas tareas, mismos datos); procesos para paralelismo grueso (tareas que conviene aislar).

Resumen



Creación y ciclo de vida

`pthread_create()`, `pthread_join()` y atributos `joinable/detached`.



Cancelación

`pthread_cancel()`, estados de cancelación y secciones críticas.



Datos específicos de hilo

Una copia por hilo con `pthread_key_create()`, más cleanup handlers.



Sincronización

Mutexes, semáforos y variables de condición evitan las condiciones de carrera.



Procesos vs. hilos

Mismos objetivos, distinto costo: paralelismo fino vs. grueso.

Referencia: Advanced Linux Programming, Capítulo 4 — Mitchell, Oldham & Samuel

FIN DEL CAPÍTULO 4 · HILOS

¿Preguntas?